

O **Query()** no **FastAPI** pode atuar de diversas formas para validar e configurar os parâmetros de query recebidos nas requisições do usuário. Aqui está uma lista de atribuições onde ele pode ser útil:

◆ 1. Definir valores obrigatórios ou opcionais

- **Obrigatório** (... no primeiro argumento)
- **Opcional** (usando `None` como padrão)

PYTHON

```
from fastapi import FastAPI, Query
from typing import Optional
```

```
app = FastAPI()
```

```
@app.get("/buscar")
async def buscar(nome: str = Query(...), idade: Optional[int] =
Query(None)):
    return {"nome": nome, "idade": idade}
```

◆ Exemplo de requisição:

- ☐ `http://127.0.0.1:8000/buscar?nome=João&idade=30`
 - ☐ `http://127.0.0.1:8000/buscar?idade=30` (Erro, pois nome é obrigatório)
-

◆ 2. Adicionar uma descrição para a documentação (Swagger)

PYTHON

```
@app.get("/produto")
async def get_produto(
    categoria: str = Query(..., description="Categoria do produto"),
    preco_min: Optional[float] = Query(None, description="Preço mínimo
para filtrar"),
):
    return {"categoria": categoria, "preco_min": preco_min}
```

◆ Benefício: Exibe descrições na **Swagger UI** (/docs).

◆ 3. Definir valores padrão

Se um valor não for enviado pelo usuário, um padrão será assumido.

PYTHON

```
@app.get("/saudacao")
async def saudacao(nome: str = Query("Visitante")):
    return {"mensagem": f"Olá, {nome}!"}
```

□ `http://127.0.0.1:8000/saudacao` → `{"mensagem": "Olá, Visitante!"}`
□ `http://127.0.0.1:8000/saudacao?nome=Lucas` → `{"mensagem": "Olá, Lucas!"}`

◆ 4. Restringir valores com (`min_length`, `max_length`, `regex`)

- `min_length` → define tamanho mínimo
- `max_length` → define tamanho máximo
- `regex` → aplica uma expressão regular

PYTHON

```
@app.get("/usuario")
async def get_usuario(
    username: str = Query(..., min_length=3, max_length=10,
    regex="^[a-zA-Z0-9]+$")
):
    return {"username": username}
```

□ `http://127.0.0.1:8000/usuario?username=User123`
□ `http://127.0.0.1:8000/usuario?username=U` → Erro (`min_length=3`)
□ `http://127.0.0.1:8000/usuario?username=Usuário!` → Erro (`regex`)

◆ 5. Definir limites numéricos (`gt`, `ge`, `lt`, `le`)

- `gt` (**greater than**) → maior que
- `ge` (**greater or equal**) → maior ou igual
- `lt` (**less than**) → menor que
- `le` (**less or equal**) → menor ou igual

PYTHON

```
@app.get("/idade")
async def validar_idade(idade: int = Query(..., gt=0, le=120)):
    return {"idade": idade}
```

□ `http://127.0.0.1:8000/idade?idade=25`
□ `http://127.0.0.1:8000/idade?idade=-5` → Erro (`gt=0`)
□ `http://127.0.0.1:8000/idade?idade=150` → Erro (`le=120`)

◆ 6. Permitir múltiplos valores (**List**)

Se o usuário pode passar vários valores no mesmo parâmetro.

PYTHON

```
from typing import List
```

```
@app.get("/tags")
async def get_tags(tags: List[str] = Query([])):
    return {"tags": tags}
```

□ `http://127.0.0.1:8000/tags?tags=python&tags=fastapi&tags=backend`

➡ □ Retorno: {"tags": ["python", "fastapi", "backend"]}

◆ 7. Definir valores permitidos (**enum**)

Restringe as opções disponíveis.

PYTHON

```
from enum import Enum
```

```
class Categoria(str, Enum):
    eletronico = "eletronico"
    livro = "livro"
    roupa = "roupa"
```

```
@app.get("/produtos")
async def listar_produtos(categoria: Categoria = Query(...)):
    return {"categoria": categoria}
```

□ `http://127.0.0.1:8000/produtos?categoria=eletronico`

□ `http://127.0.0.1:8000/produtos?categoria=alimento` → Erro (não permitido)

◆ 8. Esconder um parâmetro da documentação

Se não quiser que o parâmetro apareça na **Swagger UI**, use

`include_in_schema=False`.

PYTHON

```
@app.get("/secreto")
async def segredo(token: str = Query(..., include_in_schema=False)):
    return {"mensagem": "Acesso permitido!" if token == "123" else
            "Acesso negado!"}
```

□ `http://127.0.0.1:8000/secreto?token=123` → {"mensagem": "Acesso permitido!"}

□ `http://127.0.0.1:8000/secreto?token=abc` → {"mensagem": "Acesso negado!"}

◆ 9. Usar `deprecated=True` para avisar que um parâmetro está obsoleto

Caso um parâmetro não deva ser mais usado.

PYTHON

```
@app.get("/buscar")
async def buscar(nome: str = Query(...), apelido: Optional[str] =
Query(None, deprecated=True)):
    return {"nome": nome, "apelido": apelido}
```

Na **Swagger UI**, o `apelido` aparecerá com um aviso de obsoleto.

★ Resumo das principais funcionalidades do `Query()`

Funcionalidade	Exemplo
Definir obrigatoriedade	<code>Query(...)</code>
Adicionar descrição	<code>Query(..., description="Idade do usuário")</code>
Definir valor padrão	<code>Query(10)</code>
Limitar tamanho de string	<code>Query(..., min_length=3, max_length=10)</code>
Aplicar regex	<code>Query(..., regex="^[a-z]+\$")</code>
Restringir valores numéricos	<code>Query(..., gt=0, le=100)</code>
Aceitar múltiplos valores	<code>Query([])</code> (com <code>List</code>)
Esconder da documentação	<code>Query(..., include_in_schema=False)</code>
Marcar como obsoleto	<code>Query(..., deprecated=True)</code>